



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

DevSecOps – part 2

Michele Colajanni

Dipartimento di Informatica – Scienza e Ingegneria
michele.colajanni@unibo.it

Golden rules and best practices

TEAM

- Reduce human error (reduce stress, give pauses/distractions, ...)
- Ensure different teams are communicating and collaborating

KNOWLEDGE AND COMPETENCES

- Train your employees on cybersecurity best practices
- Train your employees on main software vulnerabilities

TOOLS

- Regularly patch hardware and software
- Code scanning, alerts, and notification of security anomalies
- Automation (scanning, discovery, and remediation of security defects)
- Visibility dashboards
- Testing



Action #1: Break down organizational silos

- To implement *DevSecOps*, organizations first need to break down the barriers between development, operations, site reliability engineering (SRE) and security teams, and make security a shared responsibility
- **Teams should align on a common set of objectives and KPIs.**
While this will inevitably involve trade-offs for all involved, prioritizing and aligning on the most important goals without adding to the security debt will help foster adoption
- Tools that enable users to work from a common source of truth are critical to supporting collaboration
 - They help break down organizational silos and supports collaboration by providing a common platform and specialized solutions for security, IT and DevOps teams
 - The shared data and reporting make it easier for teams to determine and prioritize critical tasks, build shared KPIs, track progress and iterate as needed throughout the lifecycle



Action #2: Break down applications

- Segmenting an entire software infrastructure is not easy, but it is necessary for *DevSecOps* → ***microservices***
- Once you achieve it and everything becomes well-defined with specific individual functions, it becomes easier to test and monitor each and every step of the process and make necessary changes



Action #3: Clear up responsibilities (1/2)

- *DevSecOps* is not possible by either asking your team to be more mindful about operations or letting your security team randomly jump into the development process and complain
- *DevSecOps* requires all teams to **share clear responsibility** for the security and reliability of the application and its environment — with security, development and operations teams working toward shared goals



Action #3: Clear up responsibilities (2/2)

- To practice clear responsibilities may be easier said than done because each team is driven by different priorities:
 - 1) development teams are typically focused on speed and quality of the code
 - 2) operations teams on stability and resiliency of architecture
 - 3) security teams on thoroughness, broad coverage and assurance against weaknesses or vulnerabilities
- A successful *DevSecOps* program aligns all of these goals so organizations can create applications at either the same or a faster pace with a lot more reliability and security built in



Remember (*for now and your future*)

It is not enough to be a *group* to be a *team*



Golden rules and best practices

TEAM

- Reduce human error (reduce stress, give pauses/distractions, ...)
- Ensure different teams are communicating and collaborating

KNOWLEDGE AND COMPETENCES

- Train your employees on cybersecurity best practices
- Train your employees on main software vulnerabilities

TOOLS

- Regularly patch hardware and software
- Code scanning, alerts, and notification of security anomalies
- Automation (scanning, discovery, and remediation of security defects)
- Visibility dashboards
- Testing



Security and reliability competences for *DevSecOps*

- **Security and reliability engineering**
 - Automate, Experiment, Test
- **Security operations**
 - Detect, Contain, Hunt
- **Compliance operations**
 - Assess, Manage, Respond
- **Security and reliable science**
 - Learn, Measure, Forecast, Train
- **Security basics**
 - Zoning and containment, Logging, Strong authentication, Encryption



DevSecOps: Main roles and competences

Release Manager	Planning and software lifecycle management
DevOps Lead	Jenkins, Git, Nagios, Zabbix, bit bucket, Subversion (SVN), Chef, puppet, Ansible
DevOps Automation Expert	Git, SVN, Bitbucket, Jenkins
DevOps Testing Professional	Unit testing, build testing, Selenium, MySQL
DevOps Architect	C, C++, Java, Ruby, Python
DevOps System Engineer	Linux; AWS, GCP or Azure platforms, Stress testing, Monitoring, ...
Security Professional	Risk analysis, Penetration testing



Typical enterprise ratio

- **Developers: 100**
- **Operations: 10**
- **Security: 1**

DevSecOps ratio (1 Sec and 1 Ops per team of 10 Devs)

- **Developers: 100**
- **Operations: 10**
- **Security: 10**



Security champions

They should represent the voice of security while still performing some duties as an application developer

- Security Champion developers still ship code
- Security Champion developers watch for the common security gotchas
- Security Champion developers automate security

Security champion **benefits**

- Long-term vision → Security in the design phase and throughout the whole lifecycle: top risks (severity and estimated % chance of occurring) are identified early and kept top of mind
- Productive discussions with the development team
- Understanding and empathy with the team (*trust between teams*)
- Higher level of security within the application



How to be *resilient and security-aware*: Get a PhD? Maybe, but learn the basics first!

- Design principles
- **OWASP** top vulnerabilities
- **Common Weakness Enumeration (CWE)**:
the list of software and hardware weaknesses

Organizations that fail in the basics for their applications

- Financial services: 58%
- Manufacturing: 65%
- Technology: 68%
- Healthcare: 69%
- Retail and Hospitality: 70%
- Government: 76%

[Veracode, 2020]



Secure design principles (from Saltzer and Schroeder)

1. **Economy of mechanism:** Keep the design simple
2. **Fail-safe defaults:** Fail towards denying access
3. **Complete mediation:** Check authorization of every access request
4. **Open design:** Assume attack knows the system internals
5. **Separation of privilege:** require two separate keys or other ways to check authorization (2-factor auth)
6. **Least privilege:** Give only necessary rights
7. **Least common mechanisms:** Ensure failures stay local
8. **Psychological acceptability:** design security mechanism that are easy to use and acceptable by the users



The Top 10 OWASP vulnerabilities in 2021

- 1) Broken access control
- 2) Cryptographic failures
- 3) Injection
- 4) Insecure design
- 5) Security misconfigurations
- 6) Vulnerable and outdated components
- 7) Identification and Authentication Failures
- 8) Software and Data Integrity Failures
- 9) Security logging and monitoring failures
- 10) Server-Side Request Forgery



Common Weakness Enumeration (CWE)

- **CWE™** is a community-developed list of software and hardware weakness types
- It serves as a common language, a measuring stick for security tools, and as a baseline for
 - weakness identification
 - mitigation efforts
 - prevention efforts
- **CWE™ is a living effort** with ongoing work to capture the specific effects, behaviors, exploit mechanisms, and implementation details within the CWE list as well as to review and revise the presentation approaches to provide those that best suit the community using this information



Common Weakness Enumeration (CWE)

The overall goal is to improve the quality of software with respect to known security issues within source code by

- 1) defining a unified measurable set of weaknesses
- 2) enabling effective discussions and descriptions
- 3) promoting the selection and use of software security tools and services that can find weaknesses



Example for training: Experience OWASP “Juice shop”

Juice shop is an example of an intentionally insecure app that has one of each of the OWASP top 10 vulnerabilities

- It makes a great team exercise, as it gamifies the finding of vulnerabilities and can be done in a short time
- It is enlightening to experience how intruders can exploit common vulnerabilities such as *SQL injection* and *Cross Site Scripting (XSS)*



	Apple Juice (1000ml) 1.99€		Apple Pomace 0.89€		Banana Juice (1000ml) 1.99€		Carrot Juice (1000ml) 2.99€
	Eggfruit Juice (500ml) 8.99€		Fruit Press 89.99€		Green Smoothie 1.99€		Juice Shop Adversary Trading Card (Common) 2.99€

Typical causes of software weaknesses in some areas

1. **Systems Development manager issue areas**

- What are the software weaknesses I need to protect against at the level of architecture, design, code? What have the pieces of my system (COTS packages, internal development, open source) been vetted for?
- Identify tools to vet code based on tool coverage.

2. **Assessment Tool vendors issue areas**

- Express what my tool does and identify areas I should expand coverage

3. **COTS Product vendor issue areas**

- What have I verified, and my customer want my applications is verified for?

4. **Researcher issue areas**

- Quickly understand what is known. Easily identify areas to contribute/refine/correct

5. **Educator issue areas**

- Train students with the concepts they will use in practice

6. **Operations manager issue areas**

- What issues have my applications been verified for? (COTS/Internal/OS)
- What types of issues are more critical for my technology and more likely to be successfully exploited?



Golden rules and best practices

TEAM

- Reduce human error (reduce stress, give pauses/distractions, ...)
- Ensure different teams are communicating and collaborating

KNOWLEDGE AND COMPETENCES

- Train your employees on cybersecurity best practices
- Train your employees on main software vulnerabilities

TOOLS

- Regularly patch hardware and software
- Code scanning, alerts, and notification of security anomalies
- Automation (scanning, discovery, and remediation of security defects)
- Visibility dashboards
- Testing



Adoption of new tools: A common goal

DevSecOps needs some sort of automation for speed and accuracy, thus helping to ensure that teams follow agreed upon protocols and best practices

Organizations adopting *DevSecOps* may have to add on to their existing tools and processes

- **Typical developers are not security experts.** To continue delivering software at a rapid pace, **specific security tools must be integrated** seamlessly into the existing DevOps toolchain thus allowing them to work within their familiar integrated development environment
- At the same time, **the DevOps tools** must be integrated into security workflows and operations so that security teams can be true partners to their DevOps counterparts



Tools in each phase



Design phase



- **Threat modeling** is a proactive strategy for evaluating risks. It involves identifying potential threats and developing tests or procedures to detect and respond to those threats. This involves understanding how threats may impact systems, classifying threats, and applying the appropriate security controls.
- **Threat modeling tools**
 - OWASP Threat Dragon
 - Microsoft Threat Modeling Tool
 - pytm: A Pythonic framework for threat modeling



Develop phase



- **Pre-commit hooks:** Sensitive information (e.g., access keys, access tokens, AWS keys, SSH keys, etc.) are often erroneously leaked via the public source code repositories due to accidental git commits. This can be avoided by using pre-commit hooks which check for sensitive information in the files before commits or push activity. Developers can circumvent this step easily by using **tools** such as Talisman, Git Hooks, Git Secrets, PreCommit, Detect Secrets, Git Hound
- **Secrets Management:** DevOps automation requires the storage of sensitive information (e.g., passwords, ssh keys, auth tokens, certificates, etc.) in a configuration file which can lead to exposure of credentials to an unintended audience. Secrets management allows **segregation of credentials** on a separate level and every environment can pull credentials from a specific environment and use it programmatically.
Tools: Hashicorp, Torus, Keywhiz, EnvKey, AWS Secrets Manager



Tools for vulnerability testing



- Scanning code for vulnerabilities is a basic step for securing any product
- Integrating vulnerability scanning into the entire process is an obvious place to start for implementing *DevSecOps*. This ensures that code is checked for vulnerabilities at every major stage of the delivery pipeline: from the time it is written to when it is deployed into production until operation
- Many vendors offer integration of these types of tools with *DevSecOps* platforms



Test-Driven Development approach

Write security tests that are triggered in the continuous delivery pipeline alongside unit tests

- Same benefits as **unit tests**, ensuring a baseline security in the application
- Ensures specific vulnerabilities are not (re-)opened in the future

Encourages developer ownership of application security: security vulnerability and remediation are tracked and tackled like any other task with no with no pressure, indeed even with happiness for successful “hunting”



Relevant tools for vulnerability testing

- 1) **Static Application Security Testing (SAST)** tools for detecting vulnerabilities in the code that is being developed. White-box security testing uses automated tools to perform security code review which flushes out many low-hanging fruits like SQL injection, Cross-site scripting, Deserialization vulnerabilities, and others. SAST tools are configured with generic settings, and it may need manual oversight for managing false positives. **Tools:** Semgrep, SonarQube Community, Graidit, FindSecBugs for Java, Bandit for Python
- 2) **Software Composition Analysis (SCA)** tools enable users to analyze and manage the open-source components of their applications looking for known vulnerabilities in third party components, license risk, and out-of-date libraries with built-in support for the major package-management ecosystems

OWASP also maintains an updated list of source-code analysis tools



Vulnerability scans (*white box*)

Static Code Scanner: attempt to highlight possible vulnerabilities within *static* (non-running) *source code* by using techniques such as *Taint Analysis* and *Data Flow Analysis*

- **Popular tools:** HP Fortify, Vericode, Bandit, Brakeman, Contrast
- These tools can highlight some pretty frustrating false positives, made worse by external teams who demand resolution of said vulnerabilities at all costs



Vulnerability scans (*black box*)

- **Fuzzer**: Quality assurance technique that involves providing invalid, unexpected, or random data as inputs to a computer program
 - **Popular tools**: Honggfuzz, Peach, Fuzzer, Radamsa
- **Web (vulnerability) scanner**: program which communicates with a Web application through the front-end in order to identify potential security vulnerabilities in the application and architectural weaknesses
 - **Popular tools**: Zaproxy, Cenxic, Appscan, Webinspect, Nikto



Staging phase



Scanning code for vulnerabilities before staging is another fundamental step before deploying software in real settings



Relevant tools for vulnerability testing

Dynamic Analysis Security Testing (DAST):

- Black-box/Grey-box security testing uses automated tools because SAST may not get full picture without application deployment. That is why DAST comes handy in picking out deployment specific issues
- Results from DAST and SAST can be compared to weed out false-positives. DAST tools may also need prior set of configuration settings to give good results
- ***DAST Tools:*** OWASP ZAP, Arachni, Nikto



Vulnerability Assessment

- **Vulnerability Assessment (VA):** The VA tools are able to identify various services running on a given server, probe the identified services to check if there are any vulnerabilities. It is a general practice to perform vulnerability assessment on the production systems to identify various services running in the environment and the associated vulnerabilities. It could be recommended doing VA at staging to identify any vulnerabilities before pushing moving code to the production environment.
- **Examples of VA tools**
 - OpenVAS
 - DockScan
 - Nessus



Vulnerability Management

- **Vulnerability Management (VM).** The VM tools that are used to create a *DevSecOps* pipeline that can generate plenty of vulnerability signals (each tool will have its own separate format)
 - It might become difficult to manage this data let alone track and remediate the vulnerabilities
 - Hence, VM solutions are at the core of the *DevSecOps* process where all tools are required to spool their data into those solutions so that it can be centrally managed, triaged, tracked and remediated
- **Examples of VM tools**
 - ArcherySec
 - DefectDojo
 - Faraday



Treat security vulnerabilities as quality defects

- Organizations often maintain two types of findings — **security** and **quality** issues — in two different locations
- Not only does this practice reduce visibility, but it often results in developers deprioritizing cybersecurity defects
- To address this issue, organizations **need to have security and quality findings in one place** → this provides an accurate shared view of security posture and helps the development team treat both quality and security issues with equal importance



Risk summary table

The team understands the risks their software product is exposed to by listing them, e.g.,

- | | |
|-----------------------|---|
| • Name | Zombie outbreak |
| • Severity | Extreme |
| • Chance of Occurring | Very low |
| • Mitigation | Zombie plan created last sprint in repo |
-
- | | |
|-----------------------|---|
| • Name | SQL Injection |
| • Severity | Medium |
| • Chance of Occurring | High |
| • Mitigation | Input validation enforced. Security unit tests run with deployments |



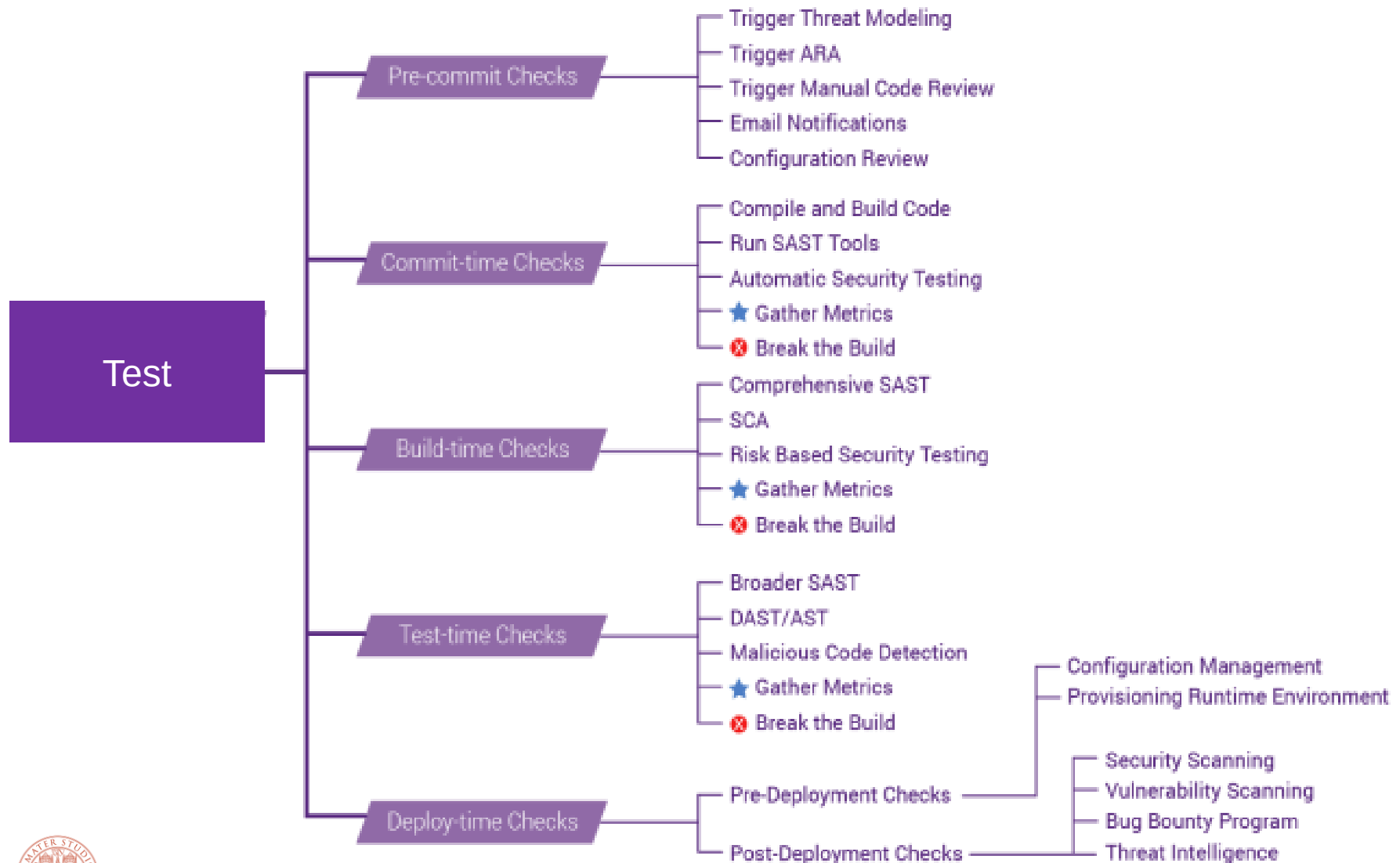
Operational phase



- **Performance monitoring.** When staging applications for testing, DevOps teams need to ensure that errors have not crept into their configuration and application performance have not degraded.
Tools: Threat Mapper, Anchore Engine, Clair
- **Altering & Monitoring:** Production applications are always faced with new threats from unknown and unforeseen vectors. This can be mitigated by having an active intrusion monitoring and prevention solution. One such open-source solution is the *Web Application Firewall* (WAF) which detects OWASP Top 10 vulnerabilities being attempted against the application.
Tools: ModSecurity, Shadow Daemon, NAXSI, Ironbee



Summary of checks



Operations



Operation: two main features

- **Performance** operation
- **Cybersecurity** operation



Performance monitoring tools

- **Performance monitoring** — When staging applications for testing, *DevSecOps* teams need to ensure that errors have not crept into their configuration and application performance have not degraded
- There are several monitoring tools that are typically related with the operational environment and containers
Possible tools: Threat Mapper, Anchore Engine, Clair



Other important tools for *performance* and *security*

Internet operations, resiliency and cybersecurity require any software that can help the team to implement and check security practices including:

- **Monitoring**
- **Log analysis**
- **Alerting**
- **Data collection** (e.g., Splunk)

NOC and SOC

These tools collect data across all tools and technology stacks. They can provide contextual visibility into the applications and the infrastructure they run on, as well as how individual stages of the process connect within the entire pipeline



Tools for continuous feedbacks in *flat* teams

- In the *DevSecOps* process, feedback should not only come from the management but from every team in the process. If the team has been properly divided into development, security, and operations, they will need to provide feedback back and forth multiple times in a day
- **The ideal approach is to develop a continuous feedback-based environment as it is essential to achieve efficiency and proficiency**
- It is this **continuous flow of information** that helps the *DevSecOps* team know where they actually stand when it comes to reliability, security and operational threats. Further, it educates the team about the latest vulnerabilities



Continuous shared visibility

- **Visibility** and **feedback** need to be contextual and end-to-end (from when a feature is defined to when it is in production) and given as the code moves through the production lane
- Both the **development** and **operations** teams need this visibility in their toolchain and existing processes, such as ticketing systems or notifications
- **Security teams should also have visibility into all relevant metrics** within their own processes and toolchains so that they can partner with their development and operations counterparts, so to have access to all information necessary for tackling any security issues that arise post-production



Runtime software protection

- **Runtime protection means securing software against threats that can arise when applications start running**
- Thinking about **scalability**, **runtime resilience** and **cybersecurity** early in the delivery process helps ensure that when deployed, many runtime threats are already mitigated. This is reason why runtime cybersecurity should be integrated across the *DevSecOps* pipeline, and not limited to the production environments
- The specific tools and strategies for runtime detection can vary depending on specific needs
 - At a minimum, it is important to ensure application monitoring for unusual behavior that could signal a breach
 - It is mandatory to be aware of which environment variables or configuration settings could create security vulnerabilities at runtime, and have a process in place for identifying and managing those risks



Automation tools for *incident management*

- When incidents occur, automation is also essential for providing visibility and simplifying remediation. Automation should be done thoughtfully, with a focus on providing accurate, actionable results not overloading systems unnecessarily, or flooding developers with false alerts
- For tasks that have to be done “out of band” and cannot be automated, each team needs to create an iterative, predetermined schedule and put a system in place that ties results into the *DevSecOps* process
- Capabilities such as risk-based alerting
 - 1) Prioritize incidents
 - 2) Reducing alert fatigue
 - 3) Facilitate cybersecurity for developers



The positive value of incidents

- Incidents help reinforce investments and/or highlight new areas that need it
- Incidents should help the application and the team to become stronger over time with effective hardening of targeted vulnerability
- Nothing like real incidents to test the strength or weakness of a service

Hope is a strategy ...

Preparation, Continuous improvement and Team collaboration is a better strategy



Strengthen post-incident response strategy

- While security issues inevitably arise in production, having full contextual visibility from when a feature was defined will help teams quickly identify the problem
- Because of the ephemeral nature of cloud architecture, it is also critical that they have full fidelity tracking on every single interaction. Even when an incident is assigned, security response and resolution teams still need to collaborate — and having shared tools and visibility will only help drive better and faster resolution
- Tools provide visibility to all incident data accompanied by built-in tools for strong incident response. That means **Site Reliability Engineering** (SRE), who are typically on the front lines, have access to all the data they need to analyze security incidents
- This allows them to route alerts to the right people, assign response, and monitor case status and progress



DevSecOps on cloud platforms



DevSecOps and Cloud Service Provider

- Another important strategy for integrating security into the application delivery process is to take advantage of the security features offered by cloud service provider(s)
- Many of the necessary tools are positioned at the deployment and post-deployment end of the DevOps chain, and thus resemble more traditional after-the-fact security services
- But they still serve an important function as part of the application's outer defenses. **Because they are part of the cloud infrastructure, they are generally easy to automate and systematize**
- Note that cloud reliability and cybersecurity features may not be enabled by default, and they may need some configuration, so you may need to take active steps in order to use them



DevSecOps on cloud platforms

AWS

<https://www.paloaltonetworks.com/prisma/environments/aws>

Azure

<https://azure.microsoft.com/en-us/solutions/devsecops/>

GCP

<https://www.reblaze.com/blog/gcp-security/https-www-reblaze-com-blog-better-devsecops-on-google-cloud-platform-gcp/>

Infosys

<https://www.infosys.com/services/agile-devops/insights/power-your-digital-transformation.pdf>



Container and service management

- Container orchestration tools (e.g., Kubernetes) have become a necessity when it comes to deploying large, container-based applications
 - Service meshes, which can work with orchestration tools and manage such things as service discovery and access, as well as the relationship between users, container-based applications, and external services, are becoming increasingly important
- Similar tools are a key element of modern *DevSecOps* at the deployment level. They act as highly scalable layers of insulation between containers and the outside world so that users and would-be attackers can only access services concealed behind proxies, for example, rather than individual containers
- Moreover, they can support such tasks as authentication, authorization, and encryption. All of them are designed for automation from the ground up



Container and service management

- Even more than with Content Security Policies tools, it is necessary to be aware of the security features that orchestration tools and service meshes provide, but you need to enable (if necessary) and configure them
- **Kubernetes' Role-based Access Configuration (RBAC)** might be a key element of *DevSecOps* under most circumstances, but it is not enabled by default



Configuration management

Platforms require some configuration management actions that is, the management of the infrastructure and of the respective configuration of all environments for an application. It consists of two parts (both relating to defining or scripting for environments):

- **Infrastructure as Code (IaC)** defines the environment including networks, servers, and other compute resources through a script that is checked into version control and used as the base source for creating or updating those infrastructures
- **Configuration as Code (CaC)** defines the configuration of servers, code, and other resources as a text file (script or definition) by managing application environment in tandem with application code. CaC is a set of processes and practices that saves time, increases flexibility, and improves system uptime because configurations are used as the base source for creating or updating those configurations



Types of configurations

- **Scripting:** writing scripts is the most direct approach to IaC
- **Configuration management tools:** also known as configuration as code, these are specialized tools designed to manage software
- **Provisioning tools:** Provisioning tools focus on creating infrastructure
- In AWS, IaC is the process of provisioning, configuring and managing cloud resources (VMs, Docker, Kubernetes) by writing a *template file* that is both human readable and machine consumable
 - **AWS CloudFormation** is the built-in choice for IaC



Compliance to standards and policies

- Setting standards and policies for security is largely a hands-on job
- You can scan your source code and infrastructure for vulnerabilities, but the process of deciding what your security priorities should be and how they should be implemented still requires serious thought on the part of human beings. The same is true of building in security standards at the design and code levels
- For example, the implementation of the EU *General Data Protection Regulation* (**GDPR**) makes it even more important for standards to be clearly formulated and put into place at the design level
- Building such standards in at the operational level can be automated to a large extent, by using orchestration tool/service mesh features, such as **Role-Based Access Control** (RBAC), to enforce policies with a high degree of granularity. The design of role-based access policies should be given as much attention as the design of security standards in application source code. They should both be regarded as high-priority tasks



Compliance as Code (CaC) – *PAY ATTENTION TO INITIALS*

- Many organisations need to apply compliance controls to their IT infrastructure to abide by industry best practices and various regulations (e.g., PCI DSS, HIPAA, SOX). On average, a regulatory compliance violation costs businesses nearly 15 million dollars
- In the past, it was possible to manage compliance manually. Today, the rapid pace of modern software delivery means that the only true solution these days is **Compliance as Code**
- By adopting an engineering mindset, it is possible to achieve compliance automatic and to focus on higher value activities
- CaC is the codification of all compliance controls so their adherence, application and remediation can be automated
- The configuration languages and formats used to write compliance as code policies vary depending on the compliance as code tools you use. Some tools have their own domain specific languages (DSLs), while others use standardised file types, like JSON



Compliance as Code (CaC) – *PAY ATTENTION TO INITIALS*

- Typically, tools work by enabling compliance stakeholders to define how IT resources must be configured in order to meet compliance controls. Then, the tools automatically scan or monitor the live IT environment and planned changes in order to detect non-compliant infrastructure
- Additionally, CaC tools often have functionality that allows them to automatically modify resources into a compliant state
- With *Infrastructure as Code* and *Configuration as Code*, the production environment is never retained. It is always torn down and re-created again and hence it is strong requirement to test the updated/newly created environment after it has been setup
- **CaC tools**
 - Inspec
 - Serverspec
 - DevSec Hardening Framework
 - Kitchen CI



The future: *DevSecOps* & AI?

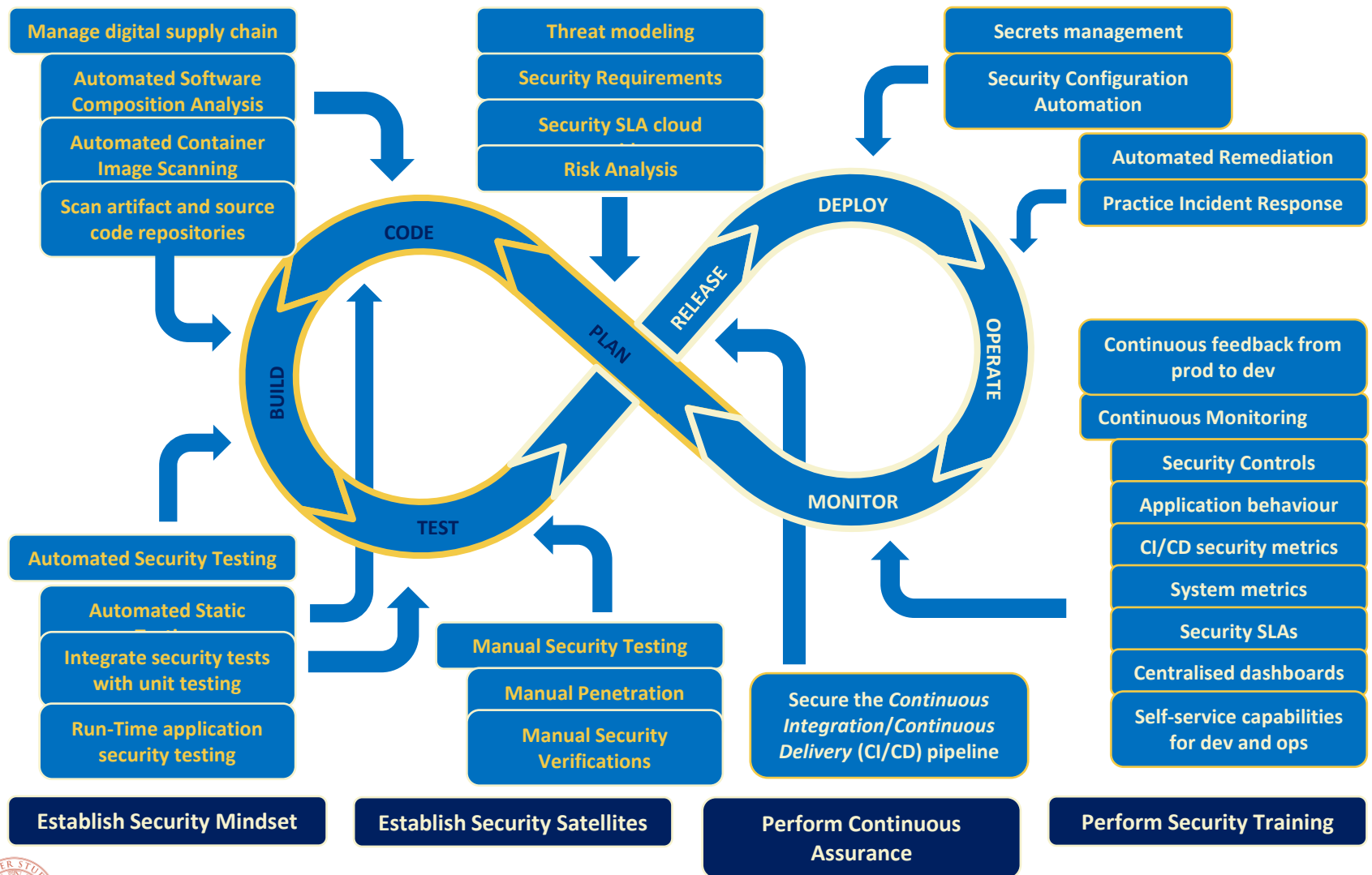


Main features of DevSecOps and AI

DevSecOps Practices	AI Solution
Generating large amount of data (often in a short period of time)	AI is the best approach to analyze in depth large amounts of data collected in logs or by tools
DevSecOps rely on static workflows and passive orchestration	AI optimizes workflows and orchestration processes based on real time data
DevSecOps monitoring – looking for what “we already know” enabling reactive actions	AI helps searching for what we do not know and prepares us to be proactive rather than reactive
SecOps teams are looking for exceptions, issues, faults (out of normal)	AI can help analyze in depth the data and shows trends and forecasting
Rule-based automation in DevOps	Self-discovering and self-governing systems
Complex feedback loop based on heterogeneous sources and tools	Faster analysis of collected data and optimized feedback loop for correction/resolution in advance



Where is it possible to apply AI to *DevSecOps*?



The need and the power of AI automation tools

- Traditional software security approaches are heavyweight, often requiring security professionals to perform all of them. *DevSecOps* requires agile and continuous feedback, whereas human approaches do not scale
- Automation tools have the power to streamline almost any business process, reduce human error and improve the agility and speed of testing. The key here is to properly apply automation whenever possible, but to verify as well
- For example, **Machine Learning** can play an important role in enhancing cybersecurity as it not only automates the essential security protocols, but also learns, evolves, and adapts to new emerging threats. In principle, tools with built-in Machine Learning supports can extract actionable insights and streamline the workflows for development, operations and security teams



Examples of AI for *DevSecOps* (1/5)

Automated software testing

- Defect identification
- Finding potential issues before we kick the testing
- Automated management of issues found in the testing process in real time
- Automated resolution of issues discovered in build, deployment, and testing
- Automated documentation and easy auditing of issues



Examples of AI for *DevSecOps* (2/5)

Process optimization

- Fine tuning of the Change Management processes and procedures for moving applications from Dev to Test to Prod environments leveraging historical data
- “Smart” management of code freeze and un-freeze
- Optimizing the Development cycle and providing proper planning based on data from delivery history of previous projects
- By evaluating some measurable aspects of the system operation (e.g., log files), ML processes can feed data into a repository that can be considered in a forthcoming release to automatically define a better approach to build the software
- All these problem have been studied by researchers working on ***self-adapting systems***. They have recently been able to perform dynamic reconfigurations of their systems through these ML techniques



Examples of AI for *DevSecOps* (3/5)

Automation of routine tasks within the CI/CD workflows

- Capturing information on build failure/success and generating and assigning automated alerts to all stakeholders; Orchestrating complex pipelines
- Automated classification and root cause analysis of build/deployment data
- Using predictive analytics to flag potential areas of concern for future builds
- Assignment of bug fixes and issues to the proper people, based on build/commit/deploy data (even from previous projects)
- Embedding security testing within the DevOps cycle
- Optimize and secure “Infrastructure as code” concepts (repeatability/security)



Examples of AI for *DevSecOps* (4a/5)

- An important challenge of AI for *DevSecOps* is the production of **glue-code** that is, custom-written programming that connects incompatible software components
- Writing a large amount of glue-code to tie all the different components of a pipeline is a time-consuming task. Most frameworks allow for some configurability, but few people have really played with the possibility of defining different paths based on metrics or analytics
- Implementing a mechanism that allows this capability may become a burden on any project that decides to play with this new trend



Examples of AI for *DevSecOps* (4b/5)

- Another scenario may present **two pipelines** that share common characteristics and use the switches to evaluate the quality of the identified ML patterns
- When the analyzed data falls into a known pattern, the ML processes can evaluate it and avoid any changes in the way the software is configured or built
- If the detected patterns show a lack of similarity with previously learned patterns, it may be time to dispatch a re-training of the models used to detect them, prior to rebuilding the software
- The co-dependence on the models and how data is shared across the two pipelines may impose challenges that are too large to resolve with COTS tools, a technical or financial burden that would prove unfeasible for many projects



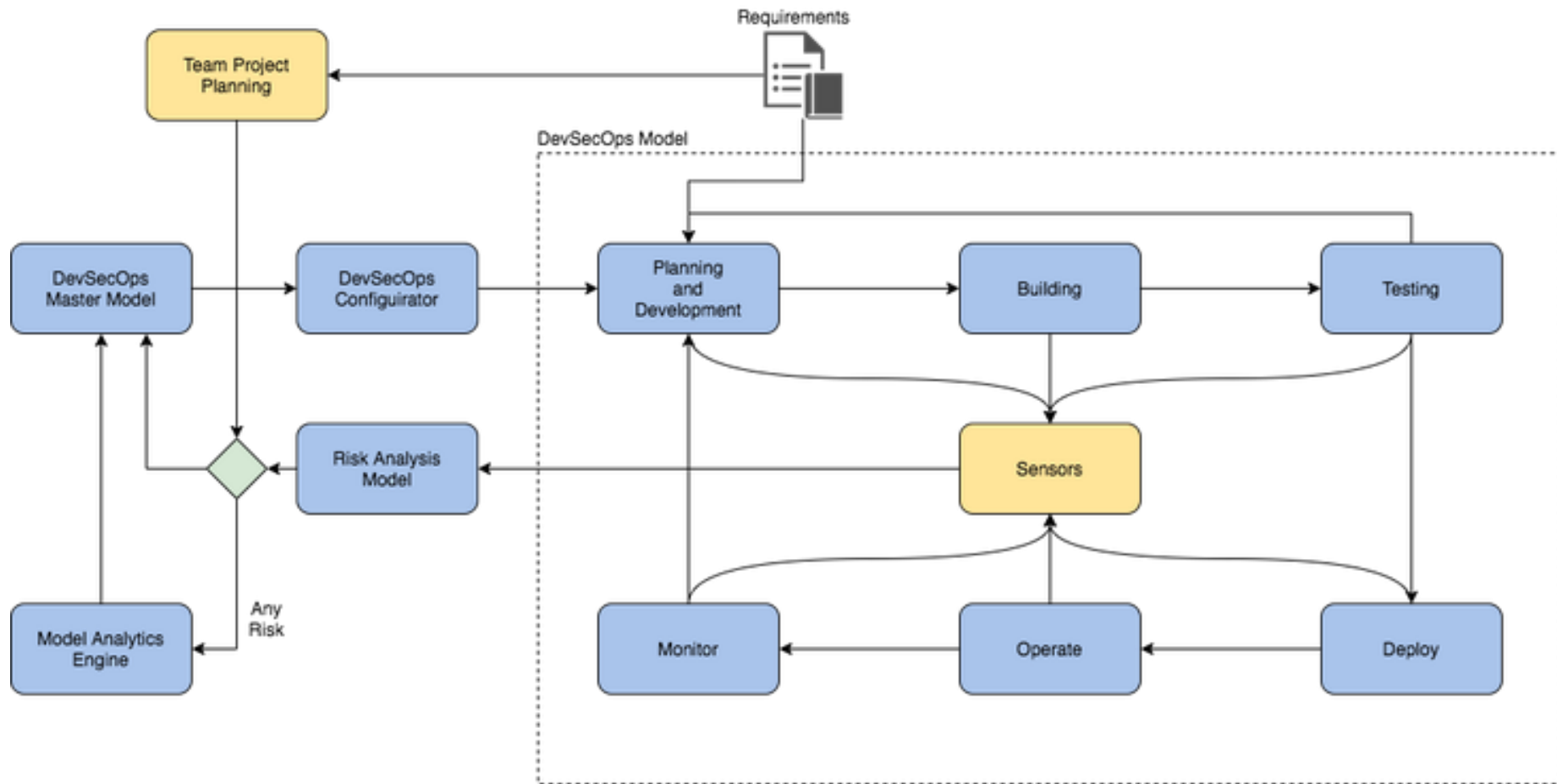
Examples of AI for *DevSecOps* (5/5)

Improved Collaboration

- Facilitate a continuous feedback loops by proactively identifying potential and real issues early on and making recommendations to address timely
- Automatic posting of issues and tagging proper people in various collaboration tools (e.g., Slack) so facilitating more effective collaboration and interactions between the Dev, Sec, and Ops teams
- Data correlation across platform and tools by analyzing data streams from various heterogeneous systems and tools to find correlations and create a holistic view of new deployments, production issues, or application's health



AI for DevSecOps (*example of application*)



AI for DevSecOps: final observation

- The fusion of *DevSecOps* and ML is a recent phenomenon
- Many different organizations are still attempting to find out what can be really achieved with this association and at what cost
- No one has a definitive position or a very strong statement about how it must be done and what the best practices are
- With that said, there are certain practices that can be proposed as a basic guidance to achieve some results in an organized way that presents minimal conflicts across the different sections of the software pipelines and their architectural characteristics



Summary



Ways to adopt *DevSecOps*

There are potentially thousands of ways to use *DevSecOps* across numerous industries and verticals. However, almost all fall into one of the following three main categories:

1. Securing the development factory
2. Supporting the creation of more secure apps
3. Securing apps already in production



1. Secure the development factory

- For developers to be successful when working within the DevOps toolchain, their environment needs to be secure and resilient. But the DevOps toolchain involves a multitude of point tools supporting various discrete functions. That complexity is further compounded by a growing reliance on open-source software for building apps, and on adopting decoupled and ephemeral architectural patterns
- There are tools (e.g., Splunk) connects telemetry across these numerous tools and analyzes data patterns using AI/ML to create readable, risk-based alerts. This helps companies ensure that their employees are adhering to security policy no matter which tools they're using, while minimizing the noise from false alerts. Some tools also supports automated incident response to simplify remediations



2. Build more secure apps

- Building more secure apps means addressing security at every layer of the app — app components, cloud services and open-source libraries that apps rely on. Addressing security at every phase also includes the app's custom code, API interactions between different services, images built and deployed, and the infrastructure — increasingly in cloud/containers — on which the code runs
- Tools can get logs from all of these layers in real-time and help track the activity pipeline from feature definition to release, and on into production security incidents. This in-depth contextual visibility is provided through shared real-time dashboards and within existing tooling
- As a result, developers can not only build more secure code, but also correct policy and security violations in real time during the process. This shared understanding across the teams also helps determine, track and measure optimal coding practices



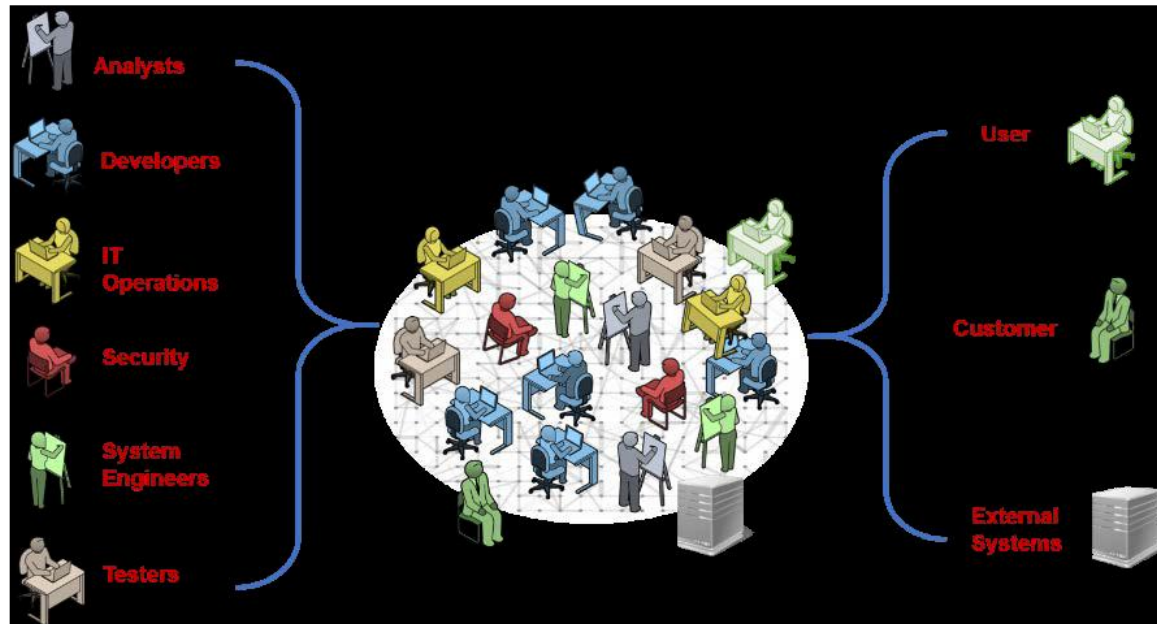
3. Secure apps in production

- When a security incident does occur after deployment, the reliability team (SRE) and cybersecurity team are typically responsible for remediation. But effectively remediating an issue is often complicated, not only due to the pace of development, but because developers need to understand both the earlier stages of the development cycle and the later stages of the in-use cycle
- Some tools track the entire activity pipeline, which accelerates both security investigation and incident resolution, and makes these processes more efficient, effectively reducing the churn between developers, SREs and security teams



The way to do software

When security becomes fully integrated with the agile pipeline of software development, DevOps and DevSecOps become similar, and that becomes simply
“This is the way we do software here”



https://resources.sei.cmu.edu/asset_files/Brochure/2018_015_001_521283.pdf

